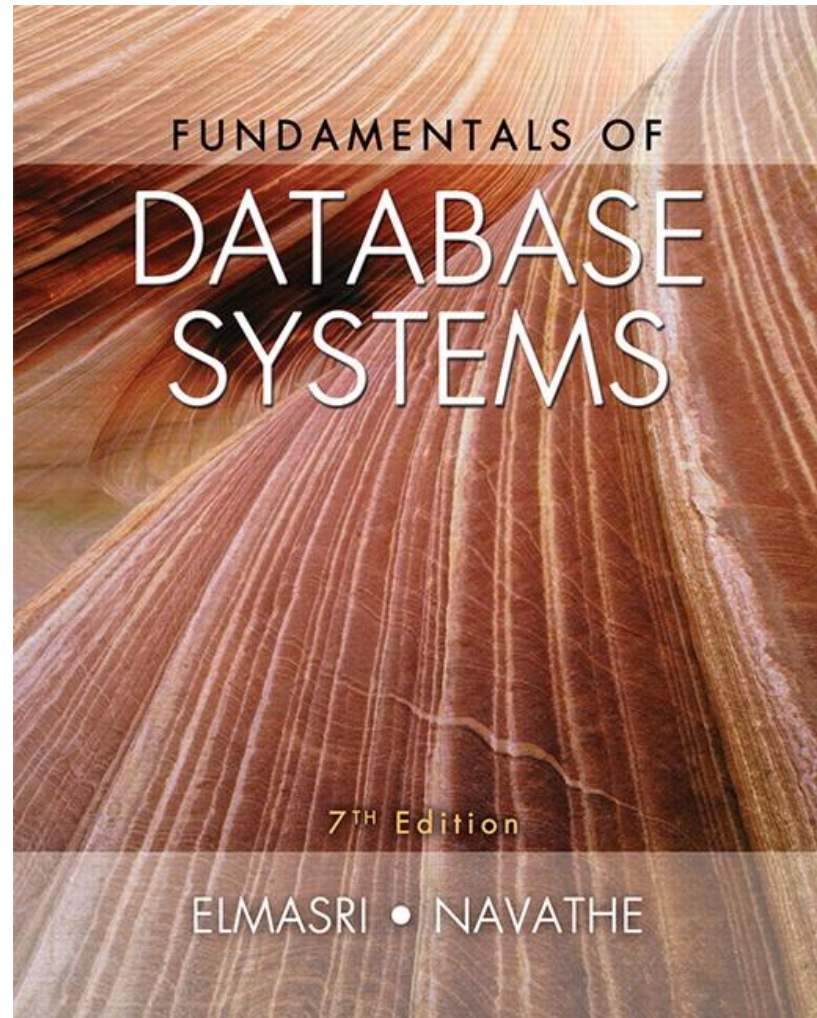




CHAPTER 24

NOSQL Databases and Big Data Storage Systems

Introduction

- NOSQL
 - Not only SQL
- Most NOSQL systems are distributed databases or distributed storage systems
 - Focus on semi-structured data storage, high performance, availability, data replication, and scalability

Introduction (cont'd.)

- NOSQL systems focus on storage of “big data”
- Typical applications that use NOSQL
 - Social media
 - Web links
 - User profiles
 - Marketing and sales
 - Posts and tweets
 - Road maps and spatial data
 - Email

24.1 Introduction to NOSQL Systems

- BigTable
 - Google's proprietary NOSQL system
 - Column-based or wide column store
- DynamoDB (Amazon)
 - Key-value data store
- Cassandra (Facebook)
 - Uses concepts from both key-value store and column-based systems

Introduction to NOSQL Systems (cont'd.)

- MongoDB and CouchDB
 - Document stores
- Neo4J and GraphBase
 - Graph-based NOSQL systems
- OrientDB
 - Combines several concepts
- Database systems classified on the object model
 - Or native XML model

Introduction to NOSQL Systems (cont'd.)

- NOSQL characteristics related to distributed databases and distributed systems
 - Scalability
 - Availability, replication, and eventual consistency
 - Replication models
 - Master-slave
 - Master-master
 - Sharding of files
 - High performance data access

Introduction to NOSQL Systems (cont'd.)

- NOSQL characteristics related to data models and query languages
 - Schema not required
 - Less powerful query languages
 - Versioning

Introduction to NOSQL Systems (cont'd.)

- Categories of NOSQL systems
 - Document-based NOSQL systems
 - NOSQL key-value stores
 - Column-based or wide column NOSQL systems
 - Graph-based NOSQL systems
 - Hybrid NOSQL systems
 - Object databases
 - XML databases

24.2 The CAP Theorem

- Various levels of consistency among replicated data items
 - Enforcing serializability the strongest form of consistency
 - High overhead – can reduce read/write operation performance
- CAP theorem
 - Consistency, availability, and partition tolerance
 - Not possible to guarantee all three simultaneously
 - In distributed system with data replication
 - Designer can choose two of three to guarantee
 - Weaker consistency level is often acceptable in NOSQL distributed data store
 - Guaranteeing availability and partition tolerance more important
 - Eventual consistency often adopted

24.3 Document-Based NOSQL Systems and MongoDB

- Documents stored as a
 - Collections of similar documents
- Individual documents resemble complex objects or XML documents
 - Documents are self-describing
 - Can have different data elements
- Documents can be specified in various formats
 - XML
 - JSON

MongoDB Data Model

- Documents stored in Binary JSON (BSON) format
- Individual documents stored in a collection
- Example command

```
db.createCollection("project", { capped : true, size : 1310720, max : 500 } )
```

- First parameter specifies name of the collection
- Collection options include limits on size and number of documents
- Each document in collection has unique ObjectID field called “_id”

MongoDB Data Model (cont'd.)

- A collection does not have a schema
 - Structure of the data fields in documents chosen based on how documents will be accessed
 - User can choose normalized or denormalized design
- Document creation using insert operation

```
db.<collection_name>.insert(<document(s)>)
```
- Document deletion using remove operation

```
db.<collection_name>.remove(<condition>)
```
- Update:
 - Uses condition like remove
 - Uses \$set clause to specify update
- Read queries: use find

```
db.<collection_name>.find(<condition>)
```

Figure 24.1 (continues)
 Example of simple documents in MongoDB (a) Denormalized document design with embedded subdocuments (b) Embedded array of document references

(a) project document with an array of embedded workers:

```
{
  _id:          "P1",
  Pname:        "ProductX",
  Plocation:    "Bellaire",
  Workers: [
    { Ename: "John Smith",
      Hours: 32.5
    },
    { Ename: "Joyce English",
      Hours: 20.0
    }
  ]
};
```

(b) project document with an embedded array of worker ids:

```
{
  _id:          "P1",
  Pname:        "ProductX",
  Plocation:    "Bellaire",
  WorkerIds:    [ "W1", "W2" ]
}

{ _id:          "W1",
  Ename:        "John Smith",
  Hours:        32.5
}

{ _id:          "W2",
  Ename:        "Joyce English",
  Hours:        20.0
}
```

(c) normalized project and worker documents (not a fully normalized design for M:N relationships):

```
{
  _id:          "P1",
  Pname:        "ProductX",
  Plocation:    "Bellaire"
}
{
  _id:          "W1",
  Ename:        "John Smith",
  ProjectId:    "P1",
  Hours:        32.5
}
```

Figure 24.1 (cont'd.)
Example of simple
documents in MongoDB
(c) Normalized documents
(d) Inserting the
documents in Figure
24.1(c) into their
collections

```
{
  _id:          "W2",
  Ename:        "Joyce English",
  ProjectId:    "P1",
  Hours:        20.0
}
```

(d) inserting the documents in (c) into their collections "project" and "worker":

```
db.project.insert( { _id: "P1", Pname: "ProductX", Plocation: "Bellaire" } )
db.worker.insert( [ { _id: "W1", Ename: "John Smith", ProjectId: "P1", Hours: 32.5 },
                    { _id: "W2", Ename: "Joyce English", ProjectId: "P1",
                      Hours: 20.0 } ] )
```

MongoDB Distributed Systems Characteristics

- Two-phase commit method
 - Used to ensure atomicity and consistency of multidocument transactions
 - Prepare then Commit
- Replication in MongoDB
 - Concept of replica set to create multiple copies on different nodes
 - Variation of master-slave approach
 - Primary copy, secondary copy, and arbiter
 - Arbiter participates in elections to select new primary if needed
 - All write operations applied to the primary copy and propagated to the secondaries
 - User can choose read preference
 - Read requests can be processed at any replica

MongoDB Distributed Systems Characteristics (cont'd.)

■ Sharding in MongoDB

- Horizontal partitioning divides the documents into disjoint partitions (shards)
- Allows adding more nodes as needed
- Multiple copies of one shard stored on different nodes to achieve load balancing
- Partitioning field (shard key) must exist in every document in the collection
 - Must have an index
- Range partitioning
 - Creates chunks by specifying a range of key values
 - Works best with range queries
- Hash partitioning
 - Partitioning based on the hash values of each shard key

24.4 NOSQL Key-Value Stores

- Key-value stores focus on high performance, availability, and scalability
 - Can store structured, unstructured, or semi-structured data
- Key: unique identifier associated with a data item
 - Used for fast retrieval
- Value: the data item itself
 - Can be string or array of bytes
 - Application interprets the structure
- No query language

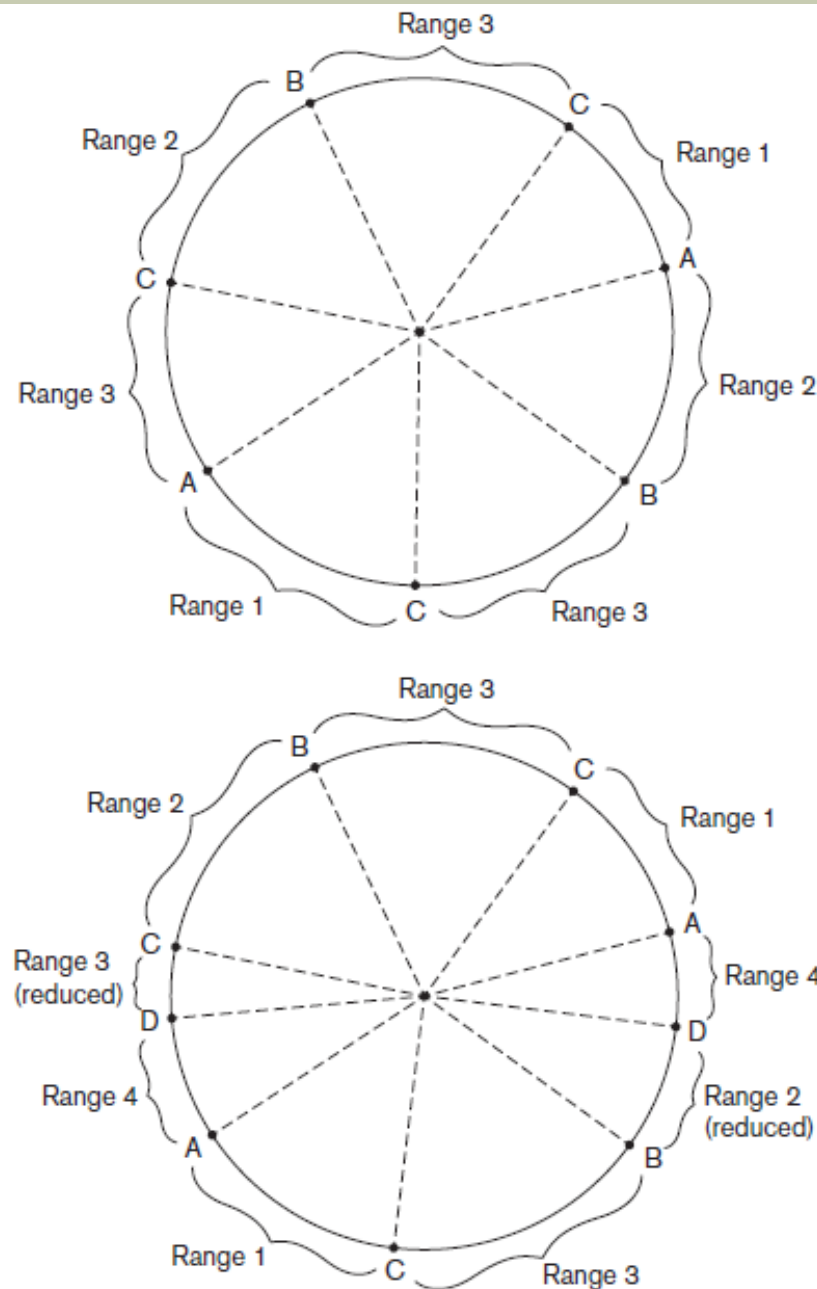
DynamoDB Overview

- DynamoDB part of Amazon's Web Services/SDK platforms
 - Proprietary
- Table holds a collection of self-describing items
- Item consists of attribute-value pairs
 - Attribute values can be single or multi-valued
- Primary key used to locate items within a table
 - Can be single attribute or pair of attributes

Voldemort Key-Value Distributed Data Store

- Voldemort: open source key-value system similar to DynamoDB
- Voldemort features
 - Simple basic operations (get, put, and delete)
 - High-level formatted data values
 - Consistent hashing for distributing (key, value) pairs
 - Consistency and versioning
 - Concurrent writes allowed
 - Each write associated with a vector clock

Figure 24.2 Example of consistent hashing
 (a) Ring having three nodes A, B, and C, with C having greater capacity. The $h(K)$ values that map to the circle points in *range 1* have their (k, v) items stored in node A, *range 2* in node B, *range 3* in node C (b) Adding a node D to the ring. Items in *range 4* are moved to the node D from node B (*range 2* is reduced) and node C (*range 3* is reduced)



Examples of Other Key-Value Stores

- Oracle key-value store
 - Oracle NOSQL Database
- Redis key-value cache and store
 - Caches data in main memory to improve performance
 - Offers master-slave replication and high availability
 - Offers persistence by backing up cache to disk
- Apache Cassandra
 - Offers features from several NOSQL categories
 - Used by Facebook and others

24.5 Column-Based or Wide Column NOSQL Systems

- BigTable: Google's distributed storage system for big data
 - Used in Gmail
 - Uses Google File System for data storage and distribution
- Apache Hbase a similar, open source system
 - Uses Hadoop Distributed File System (HDFS) for data storage
 - Can also use Amazon's Simple Storage System (S3)

Hbase Data Model and Versioning

- Data organization concepts
 - Namespaces
 - Tables
 - Column families
 - Column qualifiers
 - Columns
 - Rows
 - Data cells
- Data is self-describing

Hbase Data Model and Versioning (cont'd.)

- HBase stores multiple versions of data items
 - Timestamp associated with each version
- Each row in a table has a unique row key
- Table associated with one or more column families
- Column qualifiers can be dynamically specified as new table rows are created and inserted
- Namespace is collection of tables
- Cell holds a basic data item

(a) creating a table:

```
create 'EMPLOYEE', 'Name', 'Address', 'Details'
```

(b) Inserting some row data in the EMPLOYEE table:

```
put 'EMPLOYEE', 'row1', 'Name:Fname', 'John'
put 'EMPLOYEE', 'row1', 'Name:Lname', 'Smith'
put 'EMPLOYEE', 'row1', 'Name:Nickname', 'Johnny'
put 'EMPLOYEE', 'row1', 'Details:Job', 'Engineer'
put 'EMPLOYEE', 'row1', 'Details:Review', 'Good'
put 'EMPLOYEE', 'row2', 'Name:Fname', 'Alicia'
put 'EMPLOYEE', 'row2', 'Name:Lname', 'Zelaya'
put 'EMPLOYEE', 'row2', 'Name:MName', 'Jennifer'
put 'EMPLOYEE', 'row2', 'Details:Job', 'DBA'
put 'EMPLOYEE', 'row2', 'Details:Supervisor', 'James Borg'
put 'EMPLOYEE', 'row3', 'Name:Fname', 'James'
put 'EMPLOYEE', 'row3', 'Name:Minit', 'E'
put 'EMPLOYEE', 'row3', 'Name:Lname', 'Borg'
put 'EMPLOYEE', 'row3', 'Name:Suffix', 'Jr.'
put 'EMPLOYEE', 'row3', 'Details:Job', 'CEO'
put 'EMPLOYEE', 'row3', 'Details:Salary', '1,000,000'
```

(c) Some Hbase basic CRUD operations:

Creating a table: create <tablename>, <column family>, <column family>, ...

Inserting Data: put <tablename>, <rowid>, <column family>:<column qualifier>, <value>

Reading Data (all data in a table): scan <tablename>

Retrieve Data (one item): get <tablename>,<rowid>

Figure 24.3 Examples in Hbase (a) Creating a table called EMPLOYEE with three column families: Name, Address, and Details (b) Inserting some in the EMPLOYEE table; different rows can have different self-describing column qualifiers (Fname, Lname, Nickname, Mname, Minit, Suffix, ... for column family Name; Job, Review, Supervisor, Salary for column family Details). (c) Some CRUD operations of Hbase

Hbase Crud Operations

- Provides only low-level CRUD (create, read, update, delete) operations
- Application programs implement more complex operations
- Create
 - Creates a new table and specifies one or more column families associated with the table
- Put
 - Inserts new data or new versions of existing data items
- Get
 - Retrieves data associated with a single row
- Scan
 - Retrieves all the rows

Hbase Storage and Distributed System Concepts

- Each Hbase table divided into several regions
 - Each region holds a range of the row keys in the table
 - Row keys must be lexicographically ordered
 - Each region has several stores
 - Column families are assigned to stores
- Regions assigned to region servers for storage
 - Master server responsible for monitoring the region servers
- Hbase uses Apache Zookeeper and HDFS

24.6 NOSQL Graph Databases

- Graph databases
 - Data represented as a graph
 - Collection of vertices (nodes) and edges
 - Possible to store data associated with both individual nodes and individual edges

NOSQL Graph Databases

Neo4j

- Open source system
- Uses concepts of nodes and relationships Nodes can have labels
 - Zero, one, or several
- Both nodes and relationships can have properties
- Each relationship has a start node, end node, and a relationship type
- Properties specified using a map pattern
- Somewhat similar to ER/EER concepts
- Creating nodes in Neo4j
 - CREATE command
 - Part of high-level declarative query language Cypher
 - Node label can be specified when node is created
 - Properties are enclosed in curly brackets

Neo4j (cont'd.)

(a) creating some nodes for the COMPANY data (from Figure 5.6):

```
CREATE (e1: EMPLOYEE, {Empid: '1', Lname: 'Smith', Fname: 'John', Minit: 'B'})
CREATE (e2: EMPLOYEE, {Empid: '2', Lname: 'Wong', Fname: 'Franklin'})
CREATE (e3: EMPLOYEE, {Empid: '3', Lname: 'Zelaya', Fname: 'Alicia'})
CREATE (e4: EMPLOYEE, {Empid: '4', Lname: 'Wallace', Fname: 'Jennifer', Minit: 'S'})
...
CREATE (d1: DEPARTMENT, {Dno: '5', Dname: 'Research'})
CREATE (d2: DEPARTMENT, {Dno: '4', Dname: 'Administration'})
...
CREATE (p1: PROJECT, {Pno: '1', Pname: 'ProductX'})
CREATE (p2: PROJECT, {Pno: '2', Pname: 'ProductY'})
CREATE (p3: PROJECT, {Pno: '10', Pname: 'Computerization'})
CREATE (p4: PROJECT, {Pno: '20', Pname: 'Reorganization'})
...
CREATE (loc1: LOCATION, {Lname: 'Houston'})
CREATE (loc2: LOCATION, {Lname: 'Stafford'})
CREATE (loc3: LOCATION, {Lname: 'Bellaire'})
CREATE (loc4: LOCATION, {Lname: 'Sugarland'})
...
```

Figure 24.4 Examples in Neo4j using the Cypher language (a) Creating some nodes

Neo4j (cont'd.)

(b) creating some relationships for the COMPANY data (from Figure 5.6):

```
CREATE (e1) - [ : WorksFor ] -> (d1)
```

```
CREATE (e3) - [ : WorksFor ] -> (d2)
```

...

```
CREATE (d1) - [ : Manager ] -> (e2)
```

```
CREATE (d2) - [ : Manager ] -> (e4)
```

...

```
CREATE (d1) - [ : LocatedIn ] -> (loc1)
```

```
CREATE (d1) - [ : LocatedIn ] -> (loc3)
```

```
CREATE (d1) - [ : LocatedIn ] -> (loc4)
```

```
CREATE (d2) - [ : LocatedIn ] -> (loc2)
```

...

```
CREATE (e1) - [ : WorksOn, {Hours: '32.5'} ] -> (p1)
```

```
CREATE (e1) - [ : WorksOn, {Hours: '7.5'} ] -> (p2)
```

```
CREATE (e2) - [ : WorksOn, {Hours: '10.0'} ] -> (p1)
```

```
CREATE (e2) - [ : WorksOn, {Hours: 10.0} ] -> (p2)
```

```
CREATE (e2) - [ : WorksOn, {Hours: '10.0'} ] -> (p3)
```

```
CREATE (e2) - [ : WorksOn, {Hours: 10.0} ] -> (p4)
```

...

Figure 24.4 Examples in Neo4j using the Cypher language (b) Creating some relationships

Neo4j (cont'd.)

- Path
 - Traversal of part of the graph
 - Typically used as part of a query to specify a pattern
 - Specified by:
 - A start node
 - Followed by one or more relationships
 - Leading to one or more end node
- Schema optional in Neo4j
- Indexing and node identifiers
 - Neo4j creates a unique identifier for each node.
 - Users can create indexes for the collection of nodes that have a particular label
 - One or more properties can be indexed
- Cypher query made up of clauses
- Result from one clause can be the input to the next clause in the query

The Cypher Query Language of Neo4j

(c) Basic simplified syntax of some common Cypher clauses:

Finding nodes and relationships that match a pattern: `MATCH <pattern>`

Specifying aggregates and other query variables: `WITH <specifications>`

Specifying conditions on the data to be retrieved: `WHERE <condition>`

Specifying the data to be returned: `RETURN <data>`

Ordering the data to be returned: `ORDER BY <data>`

Limiting the number of returned data items: `LIMIT <max number>`

Creating nodes: `CREATE <node, optional labels and properties>`

Creating relationships: `CREATE <relationship, relationship type and optional properties>`

Deletion: `DELETE <nodes or relationships>`

Specifying property values and labels: `SET <property values and labels>`

Removing property values and labels: `REMOVE <property values and labels>`

Figure 24.4 Examples in Neo4j using the Cypher language (c) Basic syntax of Cypher queries

The Cypher Query Language of Neo4j (cont'd.)

- Query 1 shows how to use the MATCH and RETURN clauses in a query, and the query retrieves the locations for department number 5. Match specifies the *pattern* and the *query variables* (*d* and *loc*) and RETURN specifies the query result to be retrieved by referring to the query variables.
- Query 2 has three variables (*e*, *w*, and *p*) and returns the projects and hours per week that the employee with Empid=2 works on.
- Query 3, on the other hand, returns the employees and hours per week who work on the project with Pno = 2.
- Query 4 illustrates the ORDER BY clause and returns all employees and the projects they work on, sorted by Ename.
- It is also possible to limit the number of returned results by using the LIMIT clause as query 5 only returns the first 10 answers.

(d) Examples of simple Cypher queries:

1. MATCH (d : DEPARTMENT {Dno: '5'}) - [: LocatedIn] → (loc)
RETURN d.Dname , loc.Lname
2. MATCH (e: EMPLOYEE {Empid: '2'}) - [w: WorksOn] → (p)
RETURN e.Ename , w.Hours, p.Pname
3. MATCH (e) - [w: WorksOn] → (p: PROJECT {Pno: 2})
RETURN p.Pname, e.Ename , w.Hours
4. MATCH (e) - [w: WorksOn] → (p)
RETURN e.Ename , w.Hours, p.Pname
ORDER BY e.Ename
5. MATCH (e) - [w: WorksOn] → (p)
RETURN e.Ename , w.Hours, p.Pname
ORDER BY e.Ename
LIMIT 10
6. MATCH (e) - [w: WorksOn] → (p)
WITH e, COUNT(p) AS numOfprojs
WHERE numOfprojs > 2
RETURN e.Ename , numOfprojs
ORDER BY numOfprojs
7. MATCH (e) - [w: WorksOn] → (p)
RETURN e , w, p
ORDER BY e.Ename
LIMIT 10
8. MATCH (e: EMPLOYEE {Empid: '2'})
SET e.Job = 'Engineer'

The Cypher Query Language of Neo4j (cont'd.)

- Query 6 illustrates the use of WITH and aggregation, although the WITH clause can be used to separate clauses in a query even if there is no aggregation. Query 6 also illustrates the WHERE clause to specify additional conditions, and the query returns the employees who work on more than two projects, as well as the number of projects each employee works on.
- Query 7 is similar to query 5 but returns the nodes and relationships only, and so the query result can be displayed as a graph using Neo4j's visualization tool. It is also possible to add or remove labels and properties from nodes.
- Query 8 shows how to add more properties to a node by adding a Job property to an employee node.

(d) Examples of simple Cypher queries:

1. MATCH (d : DEPARTMENT {Dno: '5'}) - [: LocatedIn] → (loc)
RETURN d.Dname , loc.Lname
2. MATCH (e: EMPLOYEE {Empid: '2'}) - [w: WorksOn] → (p)
RETURN e.Ename , w.Hours, p.Pname
3. MATCH (e) - [w: WorksOn] → (p: PROJECT {Pno: 2})
RETURN p.Pname, e.Ename , w.Hours
4. MATCH (e) - [w: WorksOn] → (p)
RETURN e.Ename , w.Hours, p.Pname
ORDER BY e.Ename
5. MATCH (e) - [w: WorksOn] → (p)
RETURN e.Ename , w.Hours, p.Pname
ORDER BY e.Ename
LIMIT 10
6. MATCH (e) - [w: WorksOn] → (p)
WITH e, COUNT(p) AS numOfprojs
WHERE numOfprojs > 2
RETURN e.Ename , numOfprojs
ORDER BY numOfprojs
7. MATCH (e) - [w: WorksOn] → (p)
RETURN e , w, p
ORDER BY e.Ename
LIMIT 10
8. MATCH (e: EMPLOYEE {Empid: '2'})
SET e.Job = 'Engineer'

Neo4j Interfaces and Distributed System Characteristics

- Enterprise edition versus community edition
 - Enterprise edition supports caching, clustering of data, and locking
- Graph visualization interface
 - Subset of nodes and edges in a database graph can be displayed as a graph
 - Used to visualize query results
- Master-slave replication
- Caching to improve performance
- Logical logs to recover from failures

24.7 Summary

- NOSQL systems focus on storage of “big data”
- General categories
 - Document-based
 - Key-value stores
 - Column-based
 - Graph-based
 - Some systems use techniques spanning two or more categories
- Consistency paradigms
- CAP theorem